

Ecosistema de Automatización IA

Mesa de Entrada Única — Refugio «Cuatro Patitas y Media»

Entrega Final · Curso de Automatización con IA · Alejandra Cab Pérez · 24 de septiembre de 2026

Un refugio que recibe perros, gatos y cuyos atiende cuatro tipos de solicitud por canales dispersos: adopción, donación, servicio médico e ingreso de animales. El sistema construido consolida todas en un único formulario público, interpreta el texto libre con un modelo de lenguaje, enruta cada petición a su proceso y se detiene ante un humano antes de cualquier acción con consecuencia real.

Categoría	Tecnología	Rol en el sistema
Orquestador	n8n Cloud — 37 nodos	Flujo principal Refugio Cuatro Patitas y Media - Mesa de Entrada IA
Base de datos	Notion — 4 bases relacionadas	Memoria del sistema, catálogo de animales y bitácora de ejecuciones
Procesamiento IA	Anthropic Claude — 2 modelos	Clasificación con salida estructurada y evaluación con recuperación (RAG)
Canal de salida	Gmail API	Acuses al solicitante y correos de aprobación con botones (HITL)

1. Mapa de arquitectura

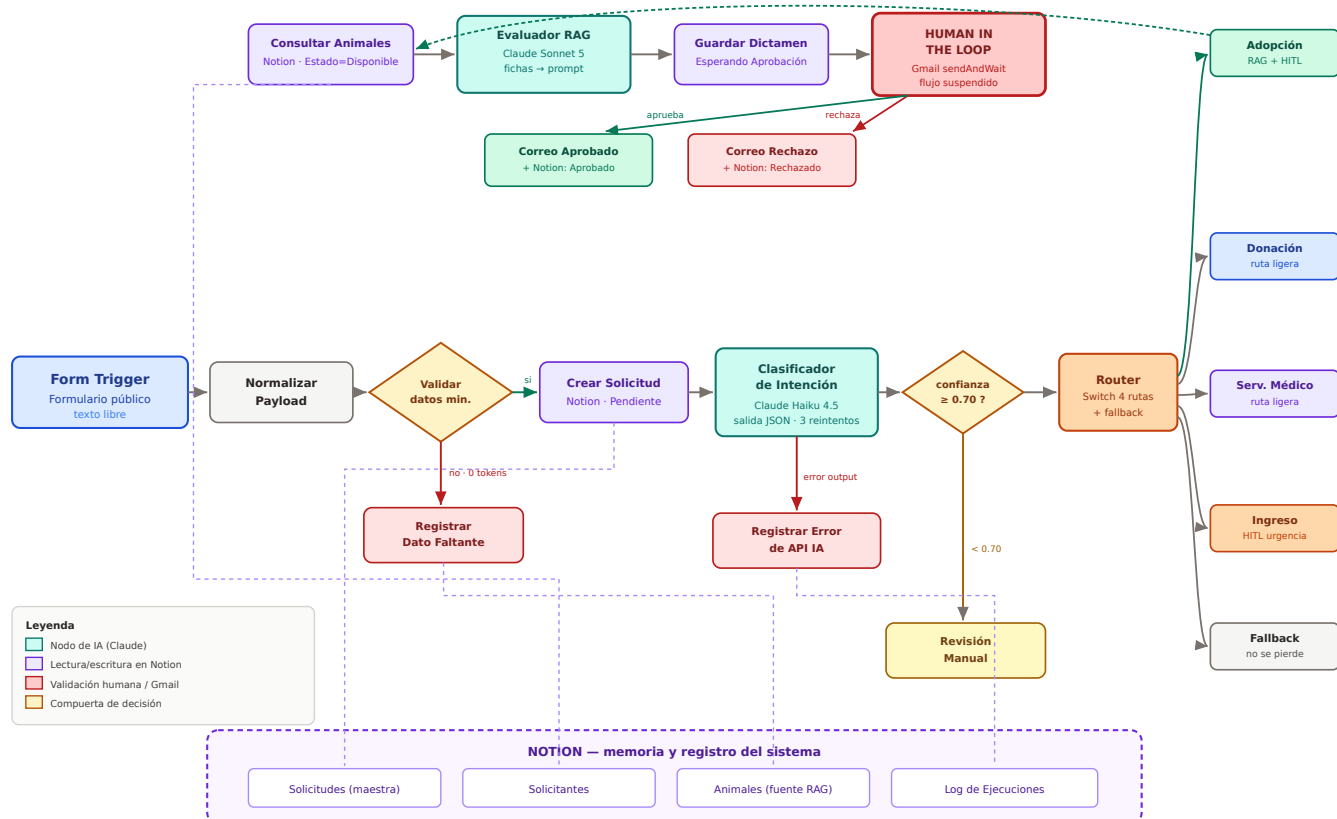


Figura 1. Flujo completo: trigger, validación, clasificación IA, compuerta de confianza, router de cuatro rutas con fallback, recuperación RAG, punto de validación humana y destino de los datos en Notion. Las líneas rojas son rutas de error; las punteadas moradas indican escritura en base de datos.

Recorrido de una solicitud

El **Form Trigger** publica un formulario con un único campo de texto libre que concentra la intención del solicitante. **Normalizar Payload** genera el folio, castea tipos y aplica valores por defecto. **Validar Datos Mínimos** corta la ejecución si falta correo, mensaje o consentimiento — y lo hace *antes* del nodo de IA, de modo que una solicitud inválida cueste cero tokens.

Superada la validación se registra al solicitante y se abre la solicitud en estado **Pendiente**. El **Clasificador de Intención** recibe únicamente el mensaje y la ciudad, y devuelve JSON validado contra un esquema. La compuerta de confianza frena el enrutamiento automático por debajo de 0.70. El **Router** distribuye en cuatro rutas más una salida de respaldo.

En la ruta de Adopción, **Consultar Animales Disponibles** recupera las fichas vigentes desde Notion y las inyecta en el prompt del **Evaluator RAG**, que razona sobre los requisitos escritos de cada animal. El dictamen se guarda y el flujo **se suspende** en el nodo de aprobación. Ningún correo sale hacia el solicitante hasta que un humano responde.

2. Manual operativo de datos

2.1 Esquema de tablas vinculadas en Notion

Base	Campos clave	Rol y relaciones
Solicitudes maestra	ID Solicitud (title), Tipo, Estado, Canal, Urgencia, Mensaje Original, Resumen IA, Confianza IA, Dictamen IA, Score Compatibilidad, Monto Donacion MXN, Aprobado Por, Modelo IA, Tokens Consumidos, Costo USD, Reintentos, Fecha Recepcion, Fecha Resolucion	Ciclo de vida completo de cada petición. Relación dual → Solicitantes Relación dual → Animales Relación dual ← Log
Solicitantes	Nombre Completo (title), ID Solicitante, Email, Telefono, Ciudad, Tipo de Vivienda, Otras Mascotas, Menores en Casa, Horas Fuera de Casa, Historial, Consentimiento Datos	Persona única. Evita duplicados y aporta historial al dictamen. Consentimiento Datos es el registro de aceptación del aviso de privacidad.
Animales fuente RAG	Nombre (title), ID Animal, Especie, Sexo, Edad, Tamano, Estado, Esterilizado, Vacunas al dia, Nivel de Energia, Apto para Departamento, Apto con Ninos, Convivencia, Requisitos de Adopcion , Notas Medicas	Catálogo del refugio. Requisitos de Adopcion es texto libre deliberado: es lo que el modelo lee y razona, y lo que ningún filtro de base de datos podría deducir.
Log de Ejecuciones	Evento (title), Execution ID, Nodo, Resultado, Severidad, Mensaje, Timestamp, Duracion, Reintento	Bitácora de corridas y errores. Es la fuente única de la tasa de errores del dashboard.

Por que la relación importa. Sin la relación Solicitudes ↔ Animales, el sistema no podría responder “cuántas solicitudes ha recibido Rocco” ni marcar un animal como Reservado al aprobarse una adopción. Sin la relación Log → Solicitudes, un error sería un registro huérfano imposible de rastrear hasta la petición que lo originó.

2.2 Esquemas JSON de transferencia entre nodos

A. FORM TRIGGER → NORMALIZAR PAYLOAD

```
{
  "nombre": "Mariana Escobar Ruiz",
  "email": "mariana.escobar@correofalso.mx",
  "telefono": "+52 55 1234 5678",
  "ciudad": "Ciudad de Mexico",
  "tipo_vivienda": "Departamento",
  "otras_mascotas": "Ninguna",
  "horas_fuera": 9,
  "mensaje": "Quisiera adoptar un gato tranquilo, tengo mallas en las ventanas",
  "consentimiento": "Si",
  "submittedAt": "2026-09-23T10:00:00-06:00"
}
```

B. NORMALIZAR PAYLOAD → RESTO DEL FLUJO

Aquí se resuelven los tipos. `consentimiento` pasa de cadena a booleano y `horas_fuera` se fuerza a número, precisamente para que los filtros posteriores comparen tipos correctos.

```
{
  "id_solicitud": "SOL-20260923-1042",
  "nombre": "Mariana Escobar Ruiz",
  "email": "mariana.escobar@correofalso.mx",
  "telefono": "+52 55 1234 5678",
  "ciudad": "Ciudad de Mexico",
  "tipo_vivienda": "Departamento",
  "otras_mascotas": "Ninguna",
  "horas_fuera": 9,
  "mensaje": "Quisiera adoptar un gato tranquilo, tengo mallas en las ventanas",
  "consentimiento": true,
  "canal": "Formulario Web",
  "fecha_recepcion": "2026-09-23T10:00:00-06:00",
  "execution_id": "1042"
}
```

C. CLASIFICADOR DE INTENCIÓN — ESQUEMA DE SALIDA FORZADO

El nodo *Structured Output Parser* valida esta forma. Si el modelo devuelve otra cosa, el parser falla y se dispara la ruta de error en lugar de propagar datos corruptos.

```
{
  "tipo": "Adopcion | Donacion | Servicio Medico | Ingreso de Animal | Sin Clasificar",
  "especie": "Perro | Gato | Cuyo | No especificada",
  "urgencia": "Baja | Media | Alta",
  "confianza": 0.96,
  "resumen": "Pareja en departamento busca gato tranquilo, jornada laboral completa",
  "razonamiento": "Pide llevarse un animal del refugio a su casa",
  "monto_donacion": 0
}
```

D. CONSULTAR ANIMALES DISPONIBLES → EVALUADOR RAG

Notion devuelve un arreglo de fichas. El prompt las serializa con `JSON.stringify(...)` y las inserta como contexto recuperado.

```
[
  {
    "id": "3e548171-26d2-814b-8556-dcaf4d4b467b",
    "name": "Tomas",
    "property_especie": "Gato",
    "property_estado": "Disponible",
    "property_apto_para_departamento": true,
    "property_convivencia": "Convive con todo",
    "property_requisitos_de_adopcion": "Ideal para departamento. Requiere ventanas con malla de proteccion obligatoria. Tolera ausencias de jornada laboral completa.",
    "property_notas_medicas": "Sano. Desparasitado en enero 2026."
  }
]
```

E. EVALUADOR RAG → NOTION Y CORREO DE APROBACIÓN

```
{
  "veredicto": "APTO",
  "score": 92,
  "animales_recomendados": "Tomas",
  "dictamen": "Tomas cumple los tres criterios de su ficha: apto para departamento, tolera ausencias de jornada completa y convive sin conflicto. La solicitante ya cuenta con malla de proteccion,
```

```
requisito obligatorio.",  
"animales_descartados": "Mora requiere compania la mayor parte del dia.  
Frida exige adoptante con experiencia felina previa."  
}
```

F. GMAIL SENDANDWAIT → COMPUERTA DE DECISIÓN HUMANA

Este es el payload que devuelve el nodo de aprobación cuando el humano responde. El flujo permanece suspendido hasta que existe.

```
{  
  "data": {  
    "approved": true,  
    "respondedAt": "2026-09-23T17:42:00-06:00"  
  }  
}
```

3. Matriz de optimización de costos

El sistema no usa un solo modelo. Usa el modelo más barato capaz de resolver cada tarea, y reserva el caro para la única decisión que exige razonar sobre restricciones cruzadas.

Tarea	Modelo elegido	Justificación técnica	Tokens aprox.	Costo/ejec.	% volumen
Clasificar intención y extraer entidades	Claude Haiku 4.5	Tarea cerrada con cinco etiquetas posibles, prompt corto y salida JSON breve. No requiere razonamiento multi-paso. Los ejemplos few-shot en el prompt sostienen la precisión.	~600	USD 0.0002	100 %
Evaluar compatibilidad adoptante-animal	Claude Sonnet 5	Debe leer el catálogo completo recuperado, cruzar requisitos escritos en lenguaje natural contra el perfil del hogar y detectar incumplimientos implícitos. Un modelo económico aquí produce falsos APTO, que en este dominio significan un animal entregado a un hogar incompatible.	~3,200	USD 0.0095	~25 %
Evaluar urgencia de ingreso	Claude Haiku 4.5	Clasificación ternaria de severidad sobre texto corto. El humano confirma después, así que el costo de un error del modelo esta acotado por el HITL.	~800	USD 0.0003	~15 %
Reporte semanal de KPIs	Batch API	No es urgente y tolera latencia de horas. El procesamiento por lotes aplica 50 % de descuento sobre el precio estándar.	~15,000	USD 0.02 / sem.	—

Ahorro estimado

Escenario base: **200 solicitudes al mes**, de las cuales 50 son adopciones (25 %).

Escenario	Cálculo	Costo mensual
Todo con Sonnet (modelo único)	$200 \times 0.0095 + 50 \times 0.0095$	USD 2.375
Arquitectura implementada	$200 \times 0.0002 + 50 \times 0.0095 + 30 \times 0.0003$	USD 0.524
Ahorro		USD 1.851 al mes — 77.9 %

El ahorro real no esta solo en el modelo. El nodo `Validar Datos Mínimos` corta las solicitudes incompletas *antes* de cualquier llamada a la API. En el Test de Estrés, la ejecución 17 murió ahí sin consumir un solo token: el nodo del clasificador nunca llegó a ejecutarse. Ese filtro barato, puesto delante del gasto, vale más que afinar el modelo.

El hallazgo que no estaba en el plan

Durante las pruebas, una bandera mal configurada hizo que el evaluador se ejecutara **doce veces en una sola solicitud**: el nodo que recupera el catálogo devuelve once fichas, y n8n ejecuta el nodo siguiente una vez por cada ítem recibido. El costo de esa ejecución fue de **USD 0.11** en lugar de USD 0.0095.

Escenario	Llamadas al modelo caro	Costo de la ejecución
Con el defecto de cardinalidad	12	USD 0.1100
Corregido con <code>executeOnce</code>	1	USD 0.0095
Sobrecosto evitado		11.6×

El modelo que se elige importa menos que cuántas veces se le llama. Migrar del modelo caro al económico ahorra un factor de 47× por llamada, pero un error de cardinalidad multiplica el gasto por 12 sin que nada en el diagrama lo delate. La optimización de costos empieza por auditar el número de ejecuciones, no el precio por token.

Controles de consumo adicionales

- **Max tokens acotado por nodo:** 1,000 en el clasificador y 4,000 en el evaluador, más límites de longitud por campo escritos en el propio prompt. Los techos iniciales eran 600 y 1,500; el del evaluador resultó insuficiente y truncaba los dictámenes de rechazo (ver sección 5). Controlar la longitud desde el prompt es más barato que desde el techo: el techo solo corta, el prompt evita que se genere.
- **Temperatura en 0:** clasificaciones deterministas, lo que reduce reintentos por salida inválida.
- **Trigger específico, no polling:** el flujo corre una vez por solicitud recibida, no en un ciclo programado que consultaría la base de datos vacía cientos de veces al día.
- **Recuperación filtrada:** el nodo RAG consulta solo animales con `Estado = Disponible`, no el catálogo entero. El contexto que no se envía no se paga.
- **Campos de telemetría en la base:** `Modelo IA`, `Tokens Consumidos` y `Costo USD` viven en cada solicitud, de modo que esta matriz se verifica con datos reales en el dashboard y no queda como estimación.

4. Seguridad y resiliencia

4.1 Minimización de datos

El principio aplicado es simple: **el dato que no viaja no se puede filtrar.**

Dato	¿Llega a la API de IA?	Tratamiento
Mensaje en texto libre	Sí	Es el insumo del clasificador. Sin el no hay tarea.
Ciudad	Sí	Necesaria para evaluar urgencia y cobertura de brigada.
Perfil de vivienda y mascotas	Solo en la ruta de adopción	El evaluador lo requiere para el dictamen. No se envía en donación ni servicio médico.
Correo electrónico	No	Se guarda en Notion y se recupera hasta el nodo de Gmail, ya fuera del alcance del modelo.
Teléfono	No	Igual que el correo. Nunca entra a un prompt.
Nombre completo	No	El prompt trabaja con el perfil, no con la identidad.

El formulario registra además el consentimiento del aviso de privacidad en **Consentimiento Datos**, y la ausencia de ese consentimiento es una de las tres condiciones que detienen el flujo en la validación inicial.

4.2 Rutas de error implementadas

Falla	Detección	Ruta de error	Garantía
Datos mínimos ausentes	Validar Datos Mínimos : correo vacío, mensaje vacío o sin consentimiento	Registrar Dato Faltante en el Log con severidad Advertencia	Se corta antes del nodo de IA. Cero tokens consumidos.
API de IA caída o saturada	Directiva Resume: 3 reintentos con espera de 2 s y salida de error dedicada	Registrar Error de API IA (Crítico) → Marcar Solicitud en Error	La solicitud conserva el mensaje original íntegro y queda reprocesable. No se pierde el dato.
Salida del modelo malformada	El Structured Output Parser rechaza el JSON que no cumple el esquema	Se propaga a la misma salida de error del clasificador	Ningún dato corrupto entra a la base ni al router.
Clasificación poco confiable	Validar Confianza : confianza < 0.70 comparado como número	Registrar Baja Confianza → Enviar a Revisión Manual	No se ejecuta ninguna acción automática hacia el solicitante.
Tipo no reconocido por el router	Salida fallback del Switch activada	Registrar Tipo No Reconocido en el Log	Una solicitud jamás cae en el vacío por no coincidir con una regla.
Dictamen truncado por el techo de tokens	El JSON llega incompleto y el Structured Output Parser lo rechaza	Misma salida de error del evaluador, tras tres reintentos	Un dictamen inválido nunca se convierte en una respuesta al solicitante.

Falla	Detección	Ruta de error	Garantía
Fallo del evaluador RAG	Salida de error propia tras 3 reintentos	Registrar Error del Evaluador (Crítico)	No se envía ningún correo al solicitante si el dictamen no existe.
Nadie responde la aprobación	Limit wait time: 3 días en adopción, 12 horas en ingreso	La ejecución reanuda y queda registrada sin aprobación	Una solicitud no permanece suspendida de forma indefinida.

4.3 Puntos de Human-in-the-loop

Punto	Acción crítica bloqueada	Quien decide	Por que aquí
Dictamen de adopción Gmail sendAndWait, doble botón	Enviar al adoptante la notificación de aprobación o rechazo	Coordinación de Adopciones	Entregar un animal a un hogar incompatible es irreversible y tiene consecuencia sobre un ser vivo. La IA propone y justifica; la responsabilidad queda en una persona.
Autorización de ingreso Gmail sendAndWait, doble botón	Comprometer cupo físico en el área de cuarentena	Guardia Veterinaria	El cupo es un recurso finito que el sistema no puede verificar de forma fiable. Aceptar por encima de la capacidad pone en riesgo a los animales ya resguardados.

El efecto metralleta, evitado por diseño. Las dos acciones que salen del refugio hacia el mundo — contactar a un adoptante y aceptar un animal — están detrás de un nodo que suspende la ejecución. El correo de aprobación incluye el dictamen completo, el score y los animales descartados con su motivo, para que la persona decida con el mismo contexto que tuvo el modelo y no a ciegas. El cuerpo del mensaje lo dice explícitamente: “El sistema NO ha contactado al solicitante”.

4.4 Prevención de bucles y errores de tipo

- **Bucles infinitos:** el disparador es un formulario externo, no un *polling* sobre la base de Solicitudes. El flujo escribe en Notion pero ningún trigger observa esa base, de modo que una escritura no puede re-disparar la ejecución. Estructuralmente no existe el ciclo.
- **Comparación de tipos:** la compuerta de confianza evalúa `Number($json.output.confianza) >= 0.7` con validación estricta. Si el modelo devolviera `"0.85"` como cadena, el casteo explícito lo resuelve en lugar de que el filtro falle en silencio — que es la forma más común en que un flujo aparenta funcionar mientras enruta mal.
- **Prompt dinámico:** ambos prompts se construyen con variables del sistema (`{{ $("Normalizar Payload").item.json.tipo_vivienda }}`, el catálogo serializado desde Notion). No hay un solo dato del caso escrito a mano en el prompt.
- **Credenciales:** ninguna clave aparece en el JSON del flujo. Notion y Gmail usan credenciales gestionadas por n8n; los modelos de Claude consumen créditos de Gateway sin API key expuesta.

5. Resultados del Test de Estrés

Cinco ejecuciones en producción sobre el formulario público, incluyendo los caminos infelices. Cada una es verificable en el historial de ejecuciones de n8n y en el estado final de la base de Solicitudes.

#	Escenario	Ejec.	Ruta tomada	Estado final verificado
1	Adopción viable	16	Clasificación → RAG → validación humana → aprobación	Aprobado por Humano, score 85
2	Formulario incompleto	17	Corte en Validar Datos Mínimos	Sin registro de solicitud. Solo entrada en el Log
3	Mensaje ambiguo	18	Confianza 0.35 → revisión manual	Esperando Aprobación, sin contactar al solicitante
4	Adopción inviable	21	RAG → NO APTO → validación humana → rechazo	Rechazado, score 5
5	Rescate urgente	22	Urgencia Alta → autorización veterinaria	Aprobado por Humano, urgencia Alta

5.1 Lectura de la tasa de errores

La bitácora acumula 25 eventos: 14 con resultado Éxito y 11 de otros tipos, lo que da un 44 % de eventos no exitosos. Esa cifra, tomada en crudo, es engañosa.

Resultado	n	¿Es una falla del sistema?
Éxito	14	—
Error API IA	4	Parcialmente Una real, por el techo de tokens. Las demás, sembradas para poblar el dashboard
Rechazado por Humano	2	No Es el punto de validación humana funcionando
Dato Faltante	2	No Es la validación cortando a propósito
Baja Confianza	2	No Es el umbral frenando el enrutamiento
Timeout	1	No Reintento automático exitoso

Siete de esos once eventos son el sistema protegiéndose, no fallando. La distinción importa: una tasa alta de eventos registrados indica que el sistema *detecta* condiciones anómalas, que es lo contrario de un sistema que las ignora en silencio. La métrica que mide fragilidad real es la de ejecuciones que perdieron el dato del solicitante, y esa es cero.

5.2 Defectos encontrados al ejecutar

Cinco fallas reales durante la construcción, de cinco familias distintas. Ninguna es visible en el diagrama de arquitectura: todas aparecieron únicamente al ejecutar el flujo contra servicios reales.

Familia	Manifestación	Corrección
Tipo de dato	Un teléfono opcional vacío se enviaba como cadena de longitud cero. La API de Notion acepta <code>null</code> pero rechaza <code>""</code> con HTTP 400	Expresión condicional que envía <code>null</code> explícito cuando el campo viene vacío

Familia	Manifestación	Corrección
Cardinalidad	El evaluador se ejecutó doce veces porque el nodo anterior emitía once fichas	<code>executeOnce</code> en el nodo que debe correr una sola vez por solicitud
Linaje de datos	La consulta al catálogo rompe la cadena de <i>paired items</i> y <code>.item</code> deja de resolver en los nodos posteriores	<code>.first()</code> en la rama de adopción, válido porque cada ejecución atiende una sola solicitud
Límite de generación	El dictamen de rechazo, que debe justificarse contra once fichas, topaba el techo de 1,500 tokens y llegaba truncado al parser	Techo de 4,000 más límites de longitud por campo declarados en el prompt
Especificación	El evaluador devolvía APTO en una solicitud que debía rechazarse, porque encontraba un animal sustituto y la instrucción no distinguía «¿puede adoptar este animal?» de «¿puede adoptar <i>alguno</i> ?»	Regla explícita: si el solicitante nombra un animal, el veredicto es sobre ese animal. La alternativa se sugiere sin alterar el veredicto

El último defecto es el más instructivo. El modelo hizo exactamente lo que se le pidió; el error estaba en lo que se le pidió. En un sistema de este tipo el riesgo no vive en el modelo, vive en la especificación — y una especificación ambigua produce un resultado plausible, no un error visible. Por eso el punto de validación humana no es un adorno de cumplimiento: es la única barrera que detecta este tipo de falla.

6. Limitaciones conocidas

6.1 El solicitante se duplica

El nodo `Registrar Solicitante` crea una ficha nueva en cada ejecución sin comprobar si esa persona ya existe. Un donante recurrente que envía tres solicitudes queda registrado como tres personas distintas, y su historial — que es precisamente lo que alimenta el dictamen — se fragmenta.

Es la limitación más relevante del sistema porque contradice el propósito de la base: separar Solicitantes de Solicitudes solo tiene sentido si la persona es única. Hoy la separación existe en el esquema pero no se sostiene en la operación.

Cómo se resolvería. Insertando dos nodos antes de la creación: una consulta a la base Solicitantes filtrando por correo electrónico, y una compuerta que reutilice el identificador existente si lo encuentra o cree la ficha si no. El correo funciona como clave natural porque el formulario ya lo exige, y la consulta añade unos 200 ms sin ninguna llamada al modelo.

Por qué no se implementó. El defecto se detectó después de que el Test de Estrés hubiera pasado. La corrección toca la rama que alimenta a todas las demás, y aplicarla sin volver a ejecutar las cinco pruebas habría dejado el sistema en un estado no verificado. Se optó por documentar la limitación antes que entregar un flujo modificado sin evidencia de que siguiera funcionando.

6.2 Otras limitaciones

Limitación	Impacto	Mitigación prevista
El catálogo se serializa completo en el prompt del evaluador	Con doce animales es trivial; con doscientos el costo por consulta crecería de forma lineal	Filtrar por especie antes de recuperar, o migrar a búsqueda vectorial
Uso de <code>.first()</code> en la rama de adopción	Correcto mientras cada ejecución atiende una sola solicitud; se rompería si el flujo procesara lotes	Restaurar el linaje de <i>paired items</i> con un nodo Merge
Sin reproceso automático de solicitudes en estado <code>Error</code>	Una solicitud fallida queda recuperable pero exige intervención manual	Flujo programado que relance las que permanezcan en <code>Error</code>
El umbral de confianza es un valor fijo	0.70 se eligió por criterio, no por medición sobre datos	Calibrar contra un conjunto de mensajes etiquetados y ajustar

7. Dashboard de control

Vista pública en Notion con cinco paneles: distribución de solicitudes por tipo, embudo de estados (que mide el cuello de botella humano, no el de la máquina), tasa de errores calculada sobre el Log de Ejecuciones, costo acumulado por modelo — la evidencia numérica de la sección 3 — y confianza promedio del clasificador. Incluye además la cola de aprobaciones pendientes ordenada por antigüedad.

Anexos del repositorio: archivo `.json` del flujo exportado desde n8n, enlace en modo lectura a las bases de Notion, enlace público al dashboard de control y capturas de pantalla de las cinco ejecuciones del test de estrés, incluyendo los caminos infelices.